O N C F M

Détection automatique de faux billets

Algorithmes de machine learning — Comparaison et analyse de confiance

Organisation Nationale de Lutte Contre le Faux-Monnayage

Contexte & Mission



L'ONCFM

Organisation publique chargée d'identifier les contrefaçons de billets en euros. Lutte quotidienne à grande échelle contre le faux-monnayage.



Le problème

Les faux billets présentent des différences géométriques infimes, indétectables à l'œil nu mais mesurables par machine.



La mission

Construire et comparer 4 algorithmes ML capables de classifier automatiquement chaque billet à partir de ses 6 dimensions géométriques.

Les données

1 500
billets labellisés

1 000
billets vrais (67%)

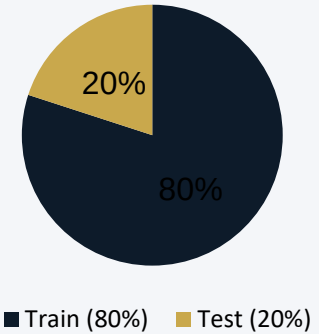
500
billets faux (33%)

6
features géométriques

Variables d'entrée

Variable	Description
diagonal	Diagonale du billet (mm)
height_left	Hauteur côté gauche (mm)
height_right	Hauteur côté droit (mm)
margin_up	Marge bord supérieur (mm)
margin_low	Marge bord inférieur (mm)
length	Longueur du billet (mm)

Découpage train / test



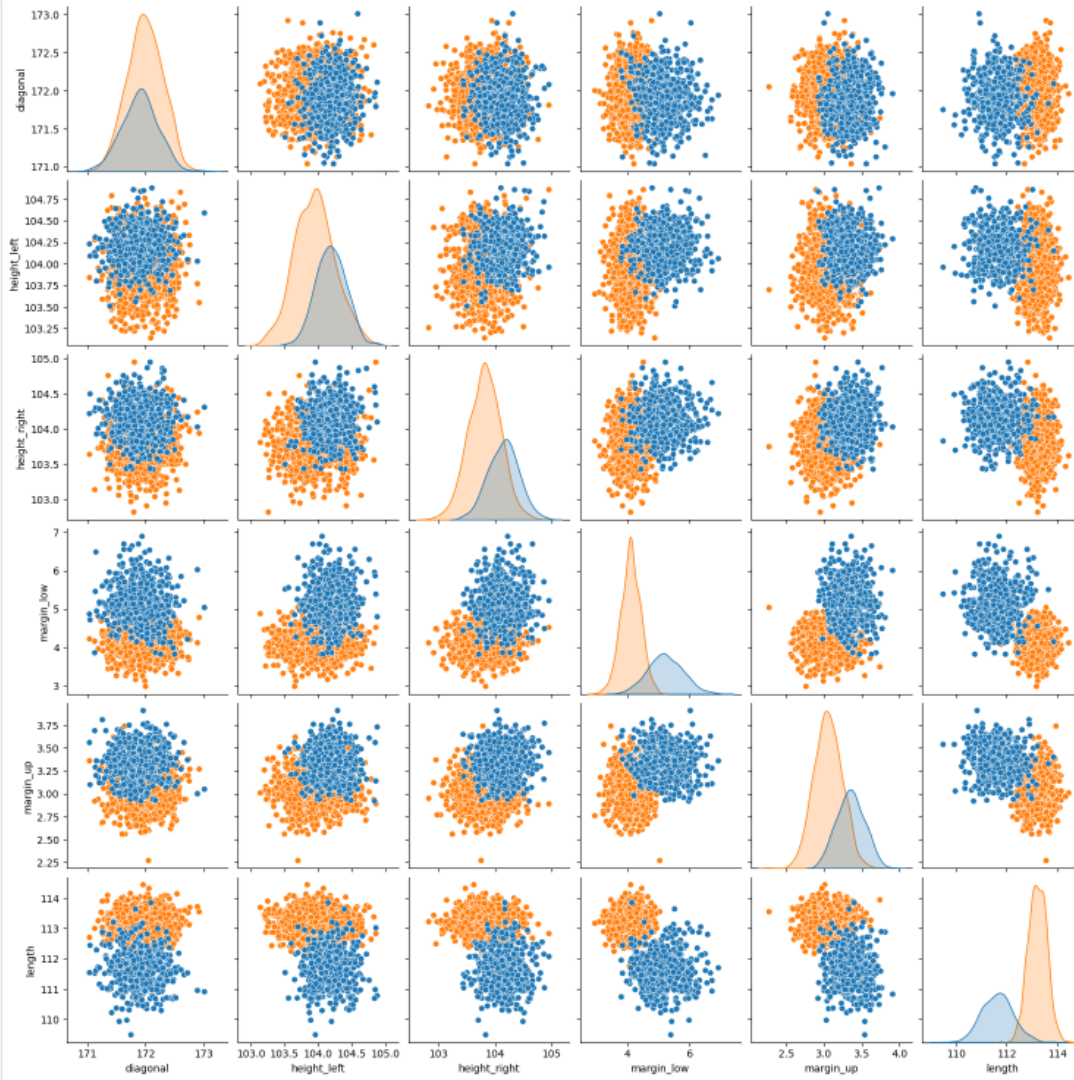
Analyse exploratoire

Imputation:
Les valeurs manquantes ont été imputées par la médiane du groupe concerné.(margin_low: 37 valeurs imputées)

Gestion des outlyers:
*L'analyse IQR révèle quelques valeurs atypiques qui ne sont pas des erreurs de mesure — maximum 2,42 IQR. **aucun outlier supprimé.***

Affichage du pair plot:
Montre des distributions relativement gaussiennes et bien séparables.

Les Corrélations montrent aussi deux nuages distincts, bien que supperposés par endroit.



Jeu d’entraînement et jeu de test : billets.csv scindé en ***train*** et ***test***. (80 %-20 %)

Les données de ***train*** ont été *standardisées* puis exporté dans **train.csv**
Les données *brutes* de ***test*** on été exportées dans **test.csv**
Le *standar scaler de train* a été exporté en **scaler.pk** pour permettre la normalisation de test.csv ou de données externes new.csv

Entrainement des moteurs et sauvegarde des paramètres

Tout les moteurs entraînés sur le même jeu d’entrainement **train.csv**
Les paramètres des moteurs on été exportés en pickle dans **nomDuMoteur.pk**

Chargement des moteur et prédiction des resultats

Le script **predict.ipynb** charge les 4 pickle **nomDuMoteur.pk** des 4 moteurs, et le fichier new.csv.
Standardisation de new.csv
Il reconstruit les moteur et prédit is_genuine sur chaque ligne de new.csv

Analyse des resulats

Les 4 algorithmes testés

📈 Régression Logistique

LR

Supervisé — linéaire

Modèle paramétrique. Frontière de décision linéaire via sigmoïde sur combinaison des 6 features. Calibré, probabilités directement interprétables.

🌲 Random Forest

RF

Supervisé — non-linéaire

Ensemble de 100 arbres de décision. Frontière complexe non-linéaire. Vote majoritaire + proportion de votes = confiance.

📍 K-Nearest Neighbors

KNN

Supervisé — local

Classifie par majorité des k=5 voisins les plus proches. Non-paramétrique, purement local. Confiances discrètes (0.6 / 0.8 / 1.0).

⭕ K-Means

KM

Non-supervisé

Clustering en 2 groupes, centroïdes assignés à une classe par vote majoritaire. Frontière quasi-linéaire, proche de LR théoriquement.

LR

RF

KNN

KMeans

📈 Régression Logistique

LR

Hyperparamètres

Paramètre	Valeur	Description
C	*1.0*	Régularisation L2 — pénalise les grands coefficients
solver	*'lbfgs'*	Optimiseur par défaut, efficace sur données denses
max_iter	*100*	Convergence atteinte sans augmentation nécessaire

🔧 Optimisation

Aucune optimisation nécessaire — convergence parfaite sur ces données. Le modèle est déterministe et stable.

RL.pkl(~2ko) — 6 coefficients + 1 intercept. Le plus compact des 4 moteurs.

📊 Indice de confiance

predict_proba() → max(p_faux, p_vrai) ∈ [0.5, 1]

Calibré par construction via la fonction sigmoïde. La confiance reflète directement la probabilité posterior de la classe prédite. C'est l'indice le plus fiable des 4 moteurs.

🌲 Random Forest

RF

Hyperparamètres

Paramètre	Valeur	Description
n_estimators	*100*	100 arbres de décision — bon compromis variance/biais
max_depth	*None*	Arbres développés jusqu'aux feuilles pures
random_state	*42*	Reproductibilité des résultats

🔧 Optimisation

Valeurs par défaut suffisantes (99% accuracy). Une optimisation via GridSearchCV sur max_depth et min_samples_split pourrait réduire le sur-apprentissage sans gain significatif ici.

RF.pkl (~446 ko) - 100 arbres complets sérialisés. Le plus volumineux : chaque arbre stocke tous ses noeuds de décision.

📊 Indice de confiance

Proportion de votes → max(p_faux, p_vrai) ∈ [0.5, 1]

Fraction des 100 arbres votant pour la classe prédite. Peu calibré — les RF ont tendance à la sur-confiance. Signal indépendant de LR car frontière non-linéaire.

📍 K-Nearest Neighbors

KNN

Hyperparamètres

Paramètre	Valeur	Description
n_neighbors	*5*	5 voisins — standard, non optimisé
metric	*'euclidean'*	Distance euclidienne dans l'espace des 6 features
weights	*'uniform'*	Tous les voisins ont le même poids

🔧 Optimisation

weights='distance' améliorerait la granularité des confiances (continues au lieu de discrètes). Non implémenté ici — le modèle est conservé pour comparaison.

knn.pkl (~141 ko) — les 1 500 points d'entraînement exportés intégralement. Pas de compression : KNN est non-paramétrique.

📊 Indice de confiance

Fraction des k voisins dans la classe prédite → {0.6, 0.8, 1.0}

Avec k=5, seulement 3 valeurs possibles : 3/5=0.6, 4/5=0.8, 5/5=1.0. Trop peu discriminant pour l'analyse de confiance → écarté de l'analyse des rangs.

⭕ K-Means

KM

Hyperparamètres

Paramètre	Valeur	Description
n_clusters	*2*	Imposé par le problème binaire vrai/faux
random_state	*52*	Reproductibilité — initialisation des centroïdes
n_init	*'auto'*	Nombre de réinitialisations automatique

🔧 Optimisation

Pas d'hyperparamètre à optimiser — 2 clusters imposés. Le mapping cluster→classe est déterminé par vote majoritaire sur le jeu d'entraînement.

kmeans.pkl (~6 Ko) — 2 centroïdes (vecteurs 6D) + mapping cluster→classe. Le plus compact avec LR.

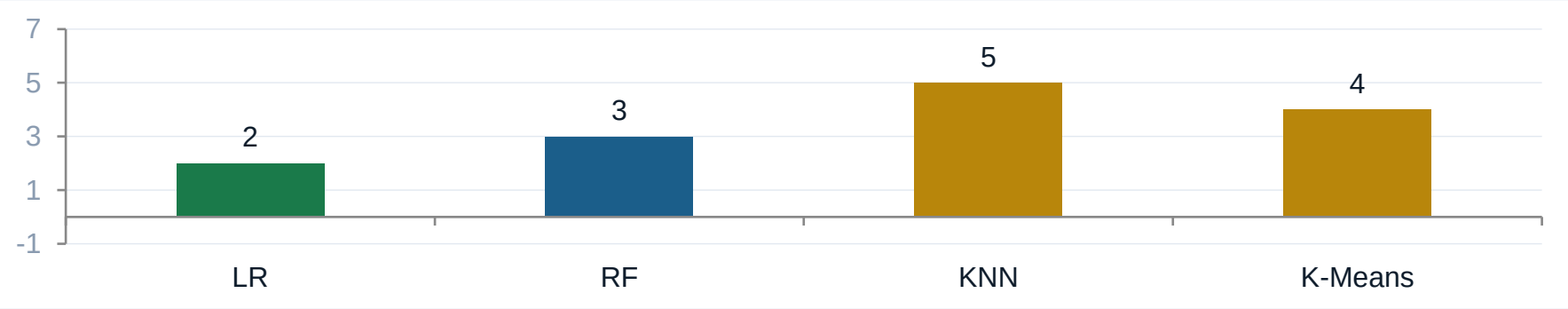
📊 Indice de confiance

d_autre / (d_prédit + d_autre) ∈ [0.5, 1]

Ratio des distances aux deux centroïdes. Approximatif — corrélé à LR (frontière quasi-linéaire). Conservé pour la diversité mais peu informatif en complément de LR.

Performances sur le jeu de test

Modèle	Accuracy	Faux positifs	Faux négatifs	Erreurs totales
Régression Logistique	99.3%	1	1	**2**
Random Forest	99.0%	2	1	**3**
KNN (k=5)	98.3%	3	2	**5**
K-Means	98.7%	2	2	**4**



Matrices de confusion

Régression Logistique

	Prédit FAUX	Prédit VRAI
Réel FAUX	99	1
Réel VRAI	1	199

Random Forest

	Prédit FAUX	Prédit VRAI
Réel FAUX	98	2
Réel VRAI	1	199

KNN

	Prédit FAUX	Prédit VRAI
Réel FAUX	97	3
Réel VRAI	2	198

K-Means

	Prédit FAUX	Prédit VRAI
Réel FAUX	98	2
Réel VRAI	2	198

Indépendance des moteurs

Les 4 moteurs ne sont pas tous indépendants — ce qui réduit la richesse du consensus :

LR ≈ K-Means (corrélés)

- K-Means à 2 clusters converge vers une frontière quasi-linéaire
- Théoriquement proche de la LDA / régression logistique (Fisher)
- Leurs erreurs sont corrélées → votes non indépendants

Random Forest (indépendant)

- Frontière non-linéaire, vote d'arbres indépendants
- Capture des interactions entre variables que LR ignore
- Structurellement différent des deux autres

✅ **En pratique : 2 moteurs indépendants réels (LR et RF) + KNN comme signal complémentaire si pondéré par distance.**

Indices de confiance par moteur

Au-delà du vote binaire vrai/faux, chaque moteur produit un score de confiance dans [0.5, 1] :

Moteur	Indice naturel	Formule	Plage	Calibré ?
LR	Probabilité posterior	max(p, 1−p)	[0.5, 1]	✅ Oui
RF	Proportion de votes	max(p_vrai, p_faux)	[0.5, 1]	⚠️ Partiel
KNN	Fraction des k voisins	k_maj / k	[0.5, 1]	❌ Discret
K-Means	Ratio de distances centroïdes	d_autre / (d_prédit + d_autre)	[0.5, 1]	⚠️ Approx.

Interprétation

- **Confiance faible (≈ 0.5)** → billet proche de la frontière de décision → à surveiller
- **Confiance élevée (≈ 1.0)** → billet bien ancré dans sa classe → prédiction fiable
- **KNN écarté** car confiances trop discrètes (0.6 / 0.8 / 1.0) → peu discriminantes

Analyse des cas ambigus

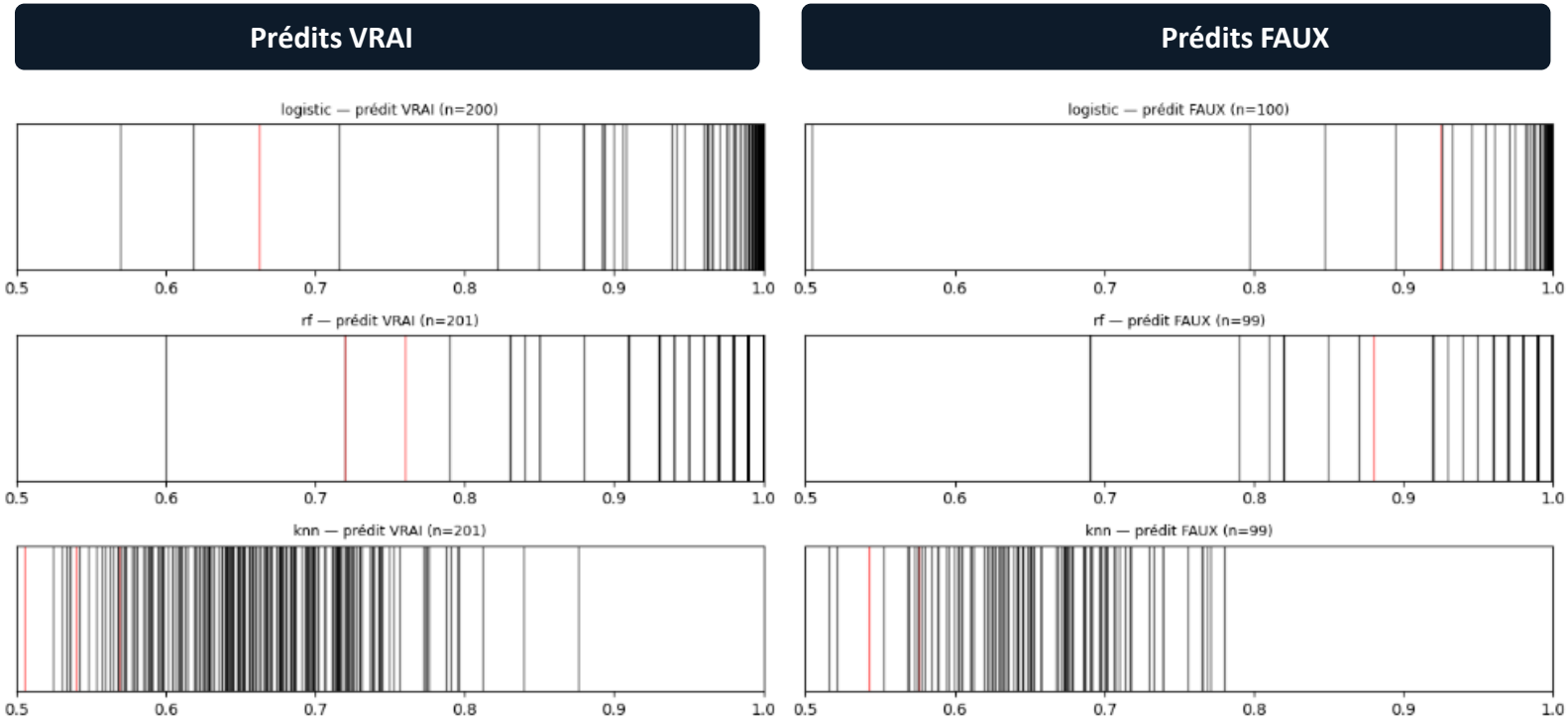
6 billets présentent un désaccord entre moteurs ou une classification inattendue :

Billet	LR	RF	KNN	KM	conf_LR	conf_RF	is_genuine	Diagnostic
206	✅ V	✅ V	✅ V	✅ V	0.66	0.72	❌ **Faux**	Outlier géométrique
65	❌ F	❌ F	❌ F	❌ F	0.93	0.88	✅ **Vrai**	Label suspect
55	✅ V	✅ V	❌ F	❌ F	0.57	0.79	✅ Vrai	Frontière
84	❌ F	✅ V	✅ V	❌ F	0.50	0.76	❌ Faux	Très ambigu
66	❌ F	❌ F	❌ F	✅ V	0.80	0.79	❌ Faux	Désaccord KM
108	❌ F	❌ F	✅ V	❌ F	0.85	0.92	❌ Faux	Désaccord KNN

Billet 206 : détectable (rang LR=3, RF=2) — faibles confiances signalent l'hésitation.
Billet 65 : indétectable — tous les moteurs très confiants dans la mauvaise direction.

Distribution des confiances — "code-barre"

Chaque trait vertical = un billet. Rouge = erreur de classification. Axe X = valeur de confiance [0.5, 1].



Rang de confiance — détection des billets suspects

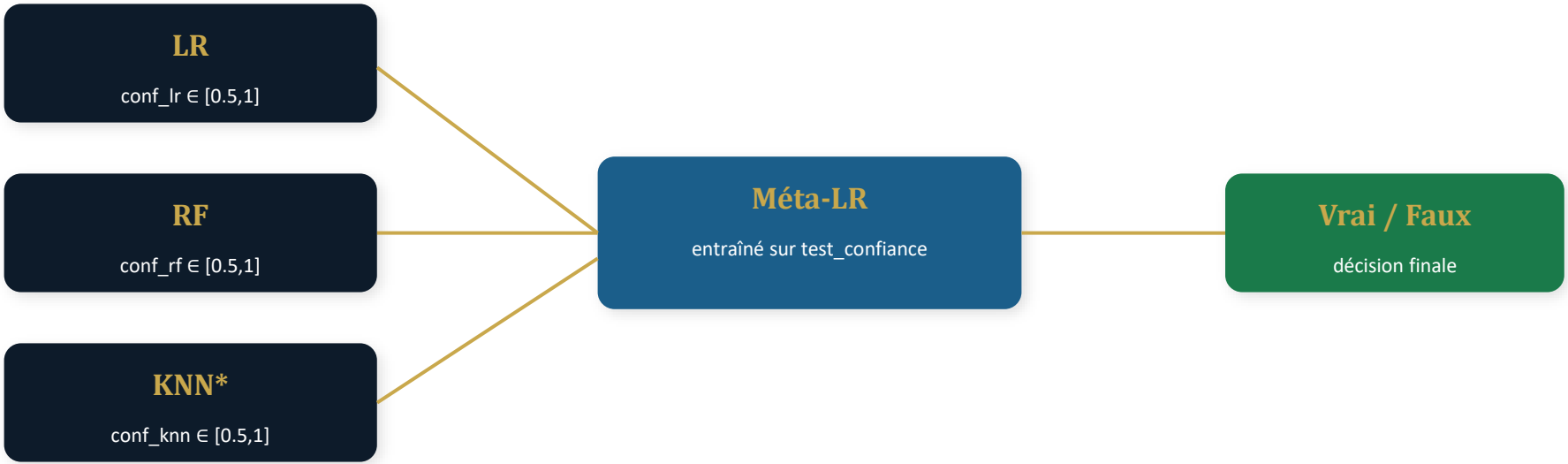
Rang 1 = billet le moins confiant dans son groupe prédit. Alerte si rang ≤ 10 (faux) ou ≤ 20 (vrais).

Billet	Rang LR	Rang RF	conf_LR	conf_RF	is_genuine	Signal
206	3	2	0.663	0.720	❌ **Faux**	⚠️ **ALERTE**
55	1	5	0.569	0.790	✅ Vrai	Frontière (correct)
179	4	1	0.894	0.690	❌ Faux	Correct (alerte)
69	9	4	0.955	0.810	❌ Faux	Correct (alerte)
65	5	9	0.925	0.880	✅ **Vrai**	❌ **Indétectable**

✅ Billet 206 : rang faible chez LR ET RF → signal d'alerte fiable. L'indice de confiance détecte correctement l'hésitation des moteurs.

❌ Billet 65 : haute confiance dans la mauvaise direction. Aucun signal interne ne peut détecter une erreur systématique unanime.

Vers un méta-modèle : le stacking



** KNN avec weights='distance' pour confiances continues*

Split requis : 3/5 train | 1/5 test_confiance (entraîne le méta-modèle) | 1/5 test_final (évalue le pipeline complet)

Méthodologie valide — mais nécessite ~10× plus de données pour un signal statistiquement exploitable.

Limites de l'approche

📊 Volume insuffisant

1 500 billets, ~10 erreurs sur le jeu de test. Pas assez de signal pour entraîner un méta-modèle stable. Il faudrait ~15 000 billets (×10) pour valider le stacking.

🔄 Corrélation LR / K-Means

Ces deux moteurs convergent théoriquement vers la même frontière linéaire. Le consensus de 4 moteurs est en réalité 2 avis indépendants (LR≈KM, RF seul différent).

⚠️ Erreur à haute confiance

Le billet 65 (prédit faux avec confiance 0.93, mais réellement vrai) est indétectable sans label externe. Peut-être un billet mislabellisé dans les données source.

📉 KNN peu informatif

Avec k=5, les confiances KNN sont discrètes (0.6 / 0.8 / 1.0) — peu discriminantes. Un KNN pondéré par distance (weights='distance') améliorerait ce point.

Modèle retenu & recommandations

🥈 Random Forest

99.0% · 3 erreurs
Frontière non-linéaire
Signal RF indépendant

🥇 Régression Logistique

99.3% accuracy · 2 erreurs
Probabilités calibrées · Interprétable
Déployable en production

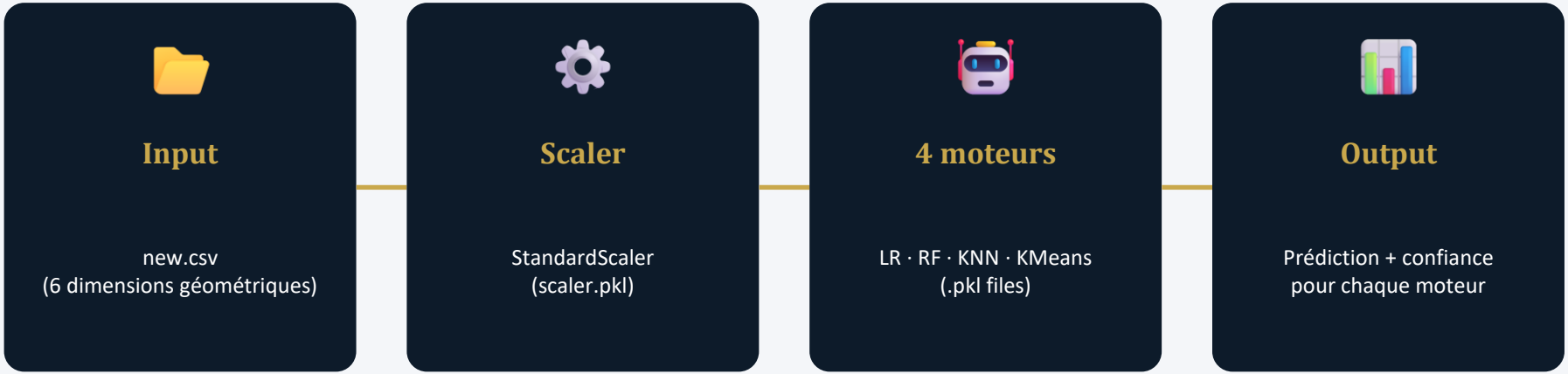
🥉 K-Means

98.7% · 4 erreurs
Non-supervisé
Validation sans labels

Recommandations

- Déployer la régression logistique comme moteur principal (confiance calibrée, déterministe)
- Utiliser RF comme moteur de vérification croisée sur les billets à faible confiance LR
- Ajouter un indicateur d'alerte : billet flaggé si confiance LR ET RF < 0.75
- Collecter davantage de données (objectif 15 000 billets) pour valider le stacking

Application fonctionnelle — predict.ipynb



Livrables

split.ipynb	Découpage stratifié 80/20 train/test
Logistic.ipynb	Entraînement + sauvegarde model_logistic.pkl
Random_Forest.ipynb	Entraînement + sauvegarde model_rf.pkl
KNN.ipynb	Entraînement + sauvegarde model_knn.pkl
K-means.ipynb	Clustering + mapping cluster→classe
predict.ipynb	Chargement modèles → prédictions + confiances + rangs

Conclusion

✅ **4 algorithmes entraînés et comparés sur 1 500 billets avec 6 features géométriques**

🏆 **La régression logistique est le modèle retenu : 99.3%, confiances calibrées, déployable**

🔍 Les indices de confiance révèlent l'hésitation des moteurs — utile pour signaler les billets ambigus

🤝 RF et LR sont les deux seuls moteurs vraiment indépendants — le vrai signal de consensus

⚠️ Erreur à haute confiance (billet 65) : indétectable sans données supplémentaires — limite fondamentale

🚀 Le stacking est la prochaine étape méthodologique — nécessite ~15 000 billets pour être statistiquement valide